Timefold Facility Location
AMPL による厳密解との比較

ロンドン（施設 30・顧客 60）インスタンス

October 27, 2026

目次

1. 問題とスコア構造

2. 数理定式化と AMPL モデル

3. 計算結果

4. まとめ

問題とスコア構造

Timefold Facility Location とは

- **出典**: Timefold Quickstarts `use-cases/facility-location`
- **対象問題**: **単一割当・容量制約付き施設配置問題**（SSCFLP）
- 各顧客（需要 15）を施設候補（容量 150）のいずれか 1 つに割り当てる
- 1 人でも顧客が割り当てられた施設は「開設」とみなされ，開設費用がかかる
- **目的**: 開設費用と顧客–施設間の距離のバランスをとる

インスタンス（ロンドン）
施設候補 30（開設費用 33,084〜72,138，容量 150），顧客 60（需要 15，総需要 900）
⇒ 1 施設あたり最大 10 顧客，最低 6 施設の開設が必要

Timefold の制約（HardSoftScore）

制約名	重み	内容
`facility capacity`	1 hard	施設の割当需要合計が容量を超えた分をペナルティ
`facility setup cost`	2 soft	使用された施設の開設費用 `setupCost`
`distance from facility`	6 soft	顧客と割当施設の距離 [m]（既定 5 を 6 に上書き）

距離の定義（`Location.getDistanceTo`）
緯度経度を平面座標とみなしたユークリッド距離に 111,000 m/度 を掛けて切り上げ

$$d_{ij} = \left\lceil 111000 \sqrt{(\phi_i - \phi_j)^2 + (\lambda_i - \lambda_j)^2} \right\rceil$$

数理定式化と AMPL モデル

0-1 整数計画としての定式化

集合・パラメータ

- J: **施設候補**，I: **顧客**
- f_j: **開設費用**，u_j: **容量**
- q_i: **需要**，d_{ij}: **距離** [m]
- $w_f = 2,\ w_d = 6$: **制約重み**

変数

- $y_j \in \{0,1\}$: **施設** j **を開設**
- $x_{ij} \in \{0,1\}$: **顧客** i **を施設** j **に割当**

$$
\begin{aligned}
\min\ & w_f \sum_j f_j y_j + w_d \sum_{i,j} d_{ij} x_{ij} \\
\text{s.t.}\ & \textstyle\sum_j x_{ij} = 1 && \forall i \\
& \textstyle\sum_i q_i x_{ij} \le u_j y_j && \forall j \\
& x_{ij} \le y_j && \forall i, j
\end{aligned}
$$

- **目的関数** $=$ Timefold **の** $-$soft **スコア**
- hard **制約（容量）は制約として厳守**
- **最終式は冗長だが** LP **緩和を強化**

AMPL モデル cflp.mod

```
set FAC;  set CUS;
param setup {FAC} >= 0;  param cap {FAC} >= 0;
param dem {CUS} >= 0;    param dist {CUS, FAC} >= 0;
param w_setup default 2; param w_dist default 6;

var y {FAC} binary;
var x {CUS, FAC} binary;

minimize SoftCost:
    w_setup * sum {j in FAC} setup[j] * y[j]
  + w_dist  * sum {i in CUS, j in FAC} dist[i,j] * x[i,j];

subject to Assign {i in CUS}:   sum {j in FAC} x[i,j] = 1;
subject to Capacity {j in FAC}: sum {i in CUS} dem[i] * x[i,j] <= cap[j] * y[j];
subject to Link {i in CUS, j in FAC}: x[i,j] <= y[j];   # 強い定式化
```

変数 $30 + 1{,}800 = 1{,}830$ **個，制約** $60 + 30 + 1{,}800 = 1{,}890$ **本**

amplpy による求解（solve_cflp.py）

```
P = json.load(open("london.json"))["problem"]          # Timefold UI の保存データ
F = {f["id"]: f for f in P["facilities"]}
C = {c["id"]: c for c in P["consumers"]}
D = {(i, j): distance(C[i]["location"], F[j]["location"]) for i in C for j in F}

ampl = AMPL(); ampl.read("cflp.mod")
ampl.set["FAC"] = list(F);  ampl.set["CUS"] = list(C)
ampl.param["setup"] = {j: F[j]["setupCost"] for j in F}
ampl.param["cap"]   = {j: F[j]["capacity"]  for j in F}
ampl.param["dem"]   = {i: C[i]["demand"]    for i in C}
ampl.param["dist"]  = D
ampl.option["solver"] = "gurobi"          # または "highs"
ampl.solve()
```

JSON に含まれる Timefold の割当（consumer.facility）からスコアを再計算し，保存値 0hard/-1510342soft と完全一致することを確認した上で比較

計算結果

Timefold 解と AMPL 最適解の比較

	Timefold	AMPL（最適）	差（AMPL−TF）
hard スコア	0	0	0
soft スコア	−1,510,342	−1,467,636	+42,706
開設施設数	7	7	0
開設費用合計	313,178	311,028	−2,150
×2（soft 寄与）	626,356	622,056	−4,300
距離合計 [m]	147,331	140,930	−6,401
×6（soft 寄与）	883,986	845,580	−38,406

- Timefold 解は最適値より 2.91% 悪い（実行可能だが最適ではない）
- 改善の約 9 割は距離項から（総距離 4.3% 短縮）
- 割当が変わった顧客: 60 人中 18 人

開設施設と割当負荷

Timefold

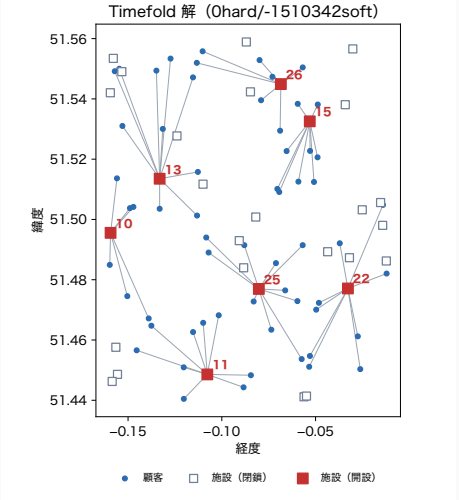
施設	開設費用	割当需要
10	42,682	90
11	46,001	135
13	46,095	150
15	43,421	135
22	33,084	135
25	54,049	150
26	47,846	105

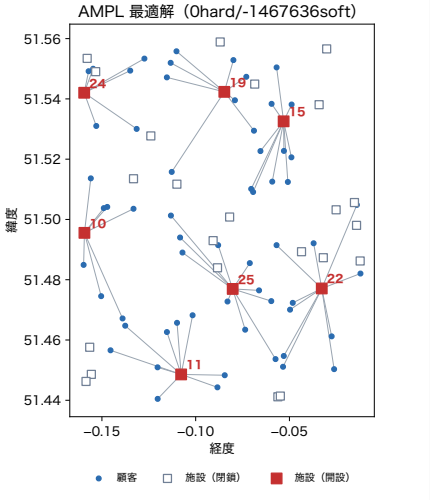
AMPL 最適解

施設	開設費用	割当需要
10	42,682	105
11	46,001	135
15	43,421	150
19	48,631	120
22	33,084	150
24	43,160	90
25	54,049	150

5 施設は共通．北部の {13, 26} を {19, 24} に入れ替えることで開設費用・距離の両方が減少

割当の可視化


Timefold 解（0hard/-1510342soft）
51.56
51.54
51.52
51.50
51.48
51.46
51.44
緯度
−0.15
−0.10
−0.05
経度
26
15
13
10
25
22
11
顧客
施設（閉鎖）
施設（開設）


AMPL 最適解（0hard/-1467636soft）
51.56
51.54
51.52
51.50
51.48
51.46
51.44
緯度
−0.15
−0.10
−0.05
経度
24
19
15
10
25
22
11
顧客
施設（閉鎖）
施設（開設）

ソルバー性能と定式化の強さ

ソルバー	定式化	LP 緩和値	LP ギャップ	求解時間 [s]
Gurobi	強（$x_{ij} \le y_j$ あり）	1,460,251.8	0.50%	0.22
Gurobi	弱（容量制約のみ）	1,145,689.0	21.94%	0.32
HiGHS 1.11	強	1,460,251.8	0.50%	1.42
HiGHS 1.11	弱	1,145,689.0	21.94%	1.26

- 全ケースで同じ最適値 1,467,636 に到達し，最適性が証明された
- 強い定式化は LP 緩和がほぼ整数解に近く，HiGHS は分枝 1 ノードで終了
- 求解時間は AMPL 呼び出しを含む実時間（ローカル環境，1 回計測）

まとめ

まとめ

- Timefold facility-location のスコアを 0-1 整数計画（SSCFLP）として厳密に再現
- 施設 30・顧客 60 の規模では AMPL + Gurobi / HiGHS が 1 秒前後で最適解を算出
- Timefold 解は実行可能（0hard）だが soft スコアで最適値から 2.91% の差
  - 開設施設 2 か所の入れ替えと 18 顧客の再割当で改善
- 冗長制約 $x_{ij} \le y_j$ により LP ギャップは 21.9% → 0.5% に縮小
- 大規模インスタンスでは Timefold（局所探索）と MIP（厳密解・下界）を併用し，下界で Timefold 解の品質を評価するのが有効

ファイル

`facility/cflp.mod`, `facility/solve_cflp.py`, `facility/london.json`, `facility/results/`